

Technical Documentation and User Manual: Gamified IMS Awareness Training

Advanced World Solutions & Advanced World Solutions Systems, Inc.

Jose Dante Chan
R&D Advisor

Julius G. Galejo Jr.
OJT Intern

May 2026

Table of Contents

I.	Project Overview.....	3
	Key Features.....	3
	Training Topics	3
	Quiz Module.....	3
	XP and Leveling Module.....	4
	Badge Module.....	5
	User Profile.....	5
II.	System Architecture.....	6
	Database	6
	Backend/Server Architecture.....	7
	Frontend/Client Architecture	10
	Development Tools	11
III.	Setup Instructions	12
	Prerequisites	12
	Step-by-step environment setup	12
IV.	Essential Skills for the Developer	14
	HTML and CSS	14
	JavaScript	14
	React.js.....	14
	Node.js	15
	NoSQL or MongoDB	15
	Version Control	15
V.	When to touch code	16
	Frontend Directory (client)	16
	Backend Directory (server)	18
	Collections Directory.....	19
VI.	Areas for Improvement.....	19
	Games Module.....	19
	Quiz Module.....	19
	Topics Module.....	19
	User accounts and email verification.....	20
VII.	A Message from the previous Developer	20

I. Project Overview

The project objective is to gamify the current IMS awareness training and introduce a new learning platform that enhances engagement and participation that, in turn, may become a solution to increase employee awareness and lessen nonconformity and violations of company policies. The system was developed as a web application that follows a server-client architecture through MERN stack.

Key Features

Training Topics

The bread and butter of the gamified system is the training topics, which encompasses the whole point for end users – to finish the IMS awareness training. Each topic consists of the following:

- a. *IMS Reference Manual* – a document that defines a policy in detail
- b. *IMS Policy Video* – a video reference that further explains a policy
- c. *Mandatory Question* – a required question to be answered to finish a topic
- d. *Bonus Questions* – 4 optional questions that can challenge a user's knowledge on the topic

Quiz Module

In order for users to participate in training, quizzes are added along with corresponding rewards. The main game of the system is the quiz module, which was mentioned above under the training topics (mandatory question and bonus questions). The quiz module follows a multiple-choice type question, fixed number of choices (4), and one correct answer per question.

- 1.1. *Mandatory Question* – Requirement for the user to answer correctly in order to finish a topic. There are prerequisites before answering the mandatory question. Firstly, the user must view all training materials (Reference manual and Policy Video). Secondly, the user must check the confirmation box that they have viewed the provided training materials. After these prerequisites,

only then will the mandatory question be available for the user. Additionally, if the user fails to answer the question correctly, then they can simply retake the mandatory question.

- 1.2. *Bonus Questions* – The bonus questions or the bonus round is a set of questions that are completely optional to answer. If the user opts for more rewards and challenge themselves on their knowledge on the topic, then they can freely answer four more questions in the bonus round.

XP and Leveling Module

Of course, the project also introduces competence; it is a game, after all. Another key feature of the system is the XP and leveling module, which will be explained in detail further in this section. For now, the basic rewards gained are as follows:

Mandatory Question: 100XP + Badge + “Completed” topic status

Bonus Questions: 50XP each correct answer

Level Thresholds: Each level requires progressively more XP.

Pre-defined thresholds (1-17):

Level 1 → 100 XP

Level 2 → 250 XP

Level 3 → 450 XP

...

Level 17 → 7600 XP

Calculation:

The last predefined threshold is 7600 XP (level 17). Once you exceed, an increment starts at 1300 and increases by 150 XP per level.

Formula:

$$XP = 7600 + (1300 + (\text{currentLevel} - 17) * 150)$$

Example:

$$\text{Level 17} \rightarrow 7600 + \{1300 + [(17 - 17) * 150]\} = 8900 \text{ XP}$$

$$\text{Level 18} \rightarrow 7600 + \{1300 + [(18 - 17) * 150]\} = 9050 \text{ XP}$$

Levels past the pre-defined thresholds (Lvl 17+) are scalable. This design ensures early levels are easier to achieve, while higher levels become progressively harder, creating a natural difficulty curve.

Badge Module

To further increase competence, badges are introduced where users can only unlock them after finishing a topic, and if you thought that was it, then you are incredibly mistaken. The badges have tiers! As you progress in the annual IMS awareness training, your badge tiers increase from bronze up to prime tier. The badges were designed in accordance with the retro-themed pixelated style of the platform.

User Profile

User settings are available in the profile tab where users can change password, logout sessions, and view profile information. Users also have the freedom to change their user avatars. However, to avoid copyright, user avatars are custom made and referenced from the company mascots. But don't fret, because the mascots are cute.

II. System Architecture

The web application was developed using MERN stack: MongoDB for the database, Express for the middleware, React JS for the frontend, and Node JS for the backend. The folder structure follows a client-server pattern, where frontend files are located in the client folder and backend files in the server folder. REST API was used for web routing and communication.

Database

MongoDB was used for the flexibility of JSON formatted data and its appropriateness with the MERN stack. For local development, the former developer used MongoDB compass to ensure continuous development even when developing offline. On the other hand, MongoDB atlas was created as an online backup and for production. The simplified database structure is as follows:

```
Users {  
  id: number,  
  username: string,  
  email: string,  
  password: string,  
  role: enum ['employee', 'admin'],  
  avatar: string (link or img src),  
  xp: number,  
  level: number,  
  completedTopics: topicId,  
  badges: topicId,  
}
```

Note: user passwords were encrypted using bcrypt.

```
Topics {  
  id: number,
```

```

    title: string,
    description: string/text,
    documentUrl: string,
    videoUrl: string,
    badgeName: string,
    badgeImage: string (link or img src),
    questions: [{
      type: string,
      options: string,
      correctAnswer: string,
      explanation: string,
      isMandatory: Boolean,
      isNew: Boolean,
      priority: number,
      isActive: boolean
    }]
  }

```

Note: the current state of the topics has a fixed number of questions (1 mandatory + 4 bonus)

The two collections mentioned above (users and topics) are the main collections for the system, other collections can be provided in the project folder "collections". If you opt to convert the database to an SQL version, then you may need to expand it to multiple tables for proper complexity of data.

Backend/Server Architecture

The current system made use of Node JS v20.17.0 LTS. For further development, it is suggested to use the latest LTS version of Node JS to avoid deprecated features and potential bugs. The current framework used for the backend are as follows:

bcryptjs v3.0 – for password hashing/encryption

cors v2.8.6 – data communication and permission

dotenv 17.2.4 – loads a .env text file for storing environment variables

express – a framework with pre-built functions for faster development of API routing and middleware

express-validator – a tool for checking sent data from express is correct

jsonwebtoken – JWT is a secure way to transfer information used for authentication; lets the server know who is logging in the system

mongoose – a framework with pre-built functions used for communicating with a MongoDB database

multer – for file uploads (used for images in logos, avatars and badges)

nodemon – Optional but highly recommended; a tool that automatically restarts the server after making changes to the code

Directory Structure:

server/

├── models/

| ├── User.js

| ├── Topic.js

| ├── Game.js

| ├── Badge.js

| ├── TrainingLog.js

| └── ActivityLog.js

├── routes/

| ├── auth.js

| ├── topics.js

| └── games.js

- | | └─ leaderboard.js
- | | └─ trainingLogs.js
- | | └─ admin.js
- | | └─ activity.js
- | └─ middleware/
- | | └─ auth.js (authenticateToken, isAdmin)
- | | └─ errorHandler.js
- | └─ utils/
- | | └─ levelSystem.js
- | | └─ badgeHelper.js
- | | └─ logger.js
- | └─ .env (not in git)
- | └─ .env.example
- | └─ .gitignore
- | └─ server.js (entry point)
- | └─ package.json

Frontend/Client Architecture

React JS was used for the frontend with its standard JS frameworks. Again, it is suggested to use the latest React.js version to avoid deprecation errors and potential bugs. The current frontend frameworks are as follows:

axios v1.13.5 – framework for communicating with the server

framer-motion v12.33.0 – react library for simple animations

react-dom v19.2.4 – This library is usually pre-built; it is an entry point that turns react elements (JS) into HTML elements

react-router-dom v7.13.0 – Another pre-built library that allows the creation of multiple components in a single page application (helpful to avoid refreshing to load contents)

Directory Structure:

```
client/
├── public/
│   ├── index.html
│   └── images/
├── src/
│   ├── components/
│   │   ├── Navbar.js
│   │   └── Footer.js
│   ├── pages/
│   │   ├── Home.js
│   │   ├── Login.js
│   │   ├── Register.js
│   │   └── Topics.js
```

```
| | └─ TopicDetail.js
| | └─ Players.js
| | └─ Profile.js
| | └─ ActivityLogs.js
| | └─ admin/
| |   └─ AdminDashboard.js
| |   └─ AdminUsers.js
| |   └─ AdminTopics.js
| └─ utils/
| └─ api.js (axios configuration)
| └─ helpers.js
└─ styles/
  └─ App.css
  └─ App.js
  └─ index.js
└─ package.json
```

Development Tools

Like any software project, a version control and a code editor was used during development. However, it is suggested to use the following tools:

Git/GitHub – version control; code maintenance and tracking

VS Code – A lightweight code editor (you can decide your preferred extensions, but do not install too many)

Postman – Optional, but helpful in terms of API testing or debugging.

III. Setup Instructions

The development setup being referred to here is assumed to be in a local network environment (localhost).

Prerequisites

- Node.js installed (version 20+ or latest LTS version)
- MongoDB Atlas account or MongoDB Compass installed
- Git installed with Git/GitHub account
- Code editor (VS Code is recommended)

Step-by-step environment setup

1. **Clone** the repository from GitHub or Extract from project zip file
2. **Install dependencies** (when entering commands, do not include the quotation marks "")
 - a. Go to the root project directory and open terminal/command line
 - b. Inside project_name> Type and enter "cd server"
 - c. Inside server> Type and enter "npm install"
 - d. After successful installation, go back to the root directory; type and enter "cd .."
 - e. This time, type and enter "cd client"
 - f. Inside client> Type and enter "npm install"
3. **Edit the .env** files for both server and client directories when needed.

Sample .env content:

PORT=5000

MONGODB_URI=mongodb://localhost:27017/database-name

4. **Run** the program

For this project, it is suggested to open three terminals while developing. One for free and Git commands, one to run the server, and another one for the client.

VS Code shortcut for opening terminals is Ctrl + Shift + ` or you can just use the command line when using Windows OS.

4.1. Server Terminal

- From the project root directory, type and enter "cd server"
- Inside server> Type and enter "npm run dev"

- Loading the server may take a while or very quick depending on your machine
- Once it loads, just leave it open and running and you can proceed to the next terminal

4.2. Client Terminal

- From the project root directory, type and enter "cd client"
- Inside client> Type and enter "npm start"
- Again, it may take a while to render all components of the react project
- Once it loads, just leave it running

5. **Access** the web app

Normally, the application automatically starts after running the server and client, but you can access the app through the following links "localhost:3000" or "127.0.0.1:3000". Alternatively, if you properly configured the .env files, then you can access it on any computer connected to the local area network via "<hostipaddress>:3000". Other issues may occur such as inbound/outbound rules for the port, but you can configure these through your computer firewall.

Rule of thumb: Since it is a web technology project, make sure to leave your console open while developing. Shortcut for most browsers is F12 or Fn + F12. This way, you can track errors easily and navigate through the components. Make it a habit to leave your console open while developing a web application.

Note: To avoid connection errors with MongoDB, it is suggested to use MongoDB Atlas. However, if opted to use MongoDB Compass, ensure to check on the following:

1. Go to C: drive or the drive where you commonly store application files
 2. Go to Program Files and find the MongoDB folder
 3. In that folder, go to Server -> version folder -> bin folder
 4. Edit the mongod.cfg (open as administrator when using notepad)
 5. Find the network interfaces, usually "bindIP"
 6. Change the ip address to 0.0.0.0 (this allows flawless connection in a local setup)
 7. Restart your mongodb service
- Go to windows and search for "Services"

- Find the MongoDB service
 - Click restart
8. Additionally, in MongoDB Compass, make sure to copy the correct connection string to be used in the .env files.

IV. Essential Skills for the Developer

Although it is not a strict need to know the exact tech stack, it is with great advantage to know the foundation of the following web technologies:

HTML and CSS

The basics of web development. It is highly suggested that you are familiar with HTML/CSS syntax and perhaps have done web projects, even if it's simple. If not, then it is a good start to familiarize or review the concepts of HTML and CSS. Another suggestion is to review Document Object Model Manipulation and HTTP methods.

JavaScript

No, you don't have to be an insane JavaScript developer nor do you have to be an experienced one to maintain this project. Aside from following proper syntax, all you have to do is review the following:

- Functions
- Async/Await
- Promises
- Arrays

That's about it, you review these and you're good to go. The rest of the work can be learned through research, reading documentation, practice, and of course using AI tools properly.

React.js

If you already know JavaScript, then this would be a cake walk. However, if you are completely new to JavaScript, then there is still no problem, because react is easy to grasp. The primary purpose of react is to apply a modular approach to web pages through the use of "components". Think of a react project as a house, where each section of the house is a component and the house as a whole is your app. For example,

the roof is a component in the app House. Other components like walls, stairs, windows, doors, etc. are components that make up a house. To further explain react.js in detail, you can always read on the following:

- Functional components
- React Hooks (useState, useEffect, etc.)
- React Router

Node.js

MERN stack revolves around one language – JavaScript. That being said, the backend makes use of the same syntax. All you have to do is review on the functions provided by Node.js like express.js. This way, you don't have to worry on the syntax and focus on how to structure the REST API and database connections. The important file in the server is the server.js or sometimes named as index.js. It is the gateway or the entry point of the middleware routes, CORS configuration, and connection handler.

NoSQL or MongoDB

If you know JSON, then creating database models in MERN will no longer be a problem. You only need to worry about the database connection issues you may encounter while developing or while shifting the project from one environment to another.

Collections import – Although you can create a new database through the system itself, you can adopt the current collections provided in the project folders. In the main project folder, there is a file named "Collections" and you can just import it on a MongoDB platform like atlas or compass.

Version Control

Commonly Git/GitHub, it is an important tool to have because not only can you track and manage changes to the code, but also serve as a secondary repository or as backup in case of data loss.

V. When to touch code

In this section of the manual, list of the files under the server and client directories will be provided with their corresponding purpose. This is to allow the developer to be guided on how to find the correct files when debugging or changing contents overall. In the folders client/public and server/uploads, you can see assets used for the webpages, which are mostly image sources. These images are used for the avatars, logo, and badges. If the developer opts to change, add, or delete assets, then these folders are all they need to find. Managing assets may not reflect immediately since image dimensions and styles are hardcoded directly in the source files. Provided below are files that may need code changes when managing content alterations. Build files and node modules will not be included in the list, meaning you will almost never touch those files.

Frontend Directory (client)

- a. client/public/avatars – folder that holds all avatar images (note that changing the images won't immediately reflect in the profile page, you still have to align the image file names in client/src/pages/Profile.js)
- b. client/public/imgrefs – folder that holds all carousel images (align image file names in client/src/pages/Home.js)
- c. client/public/index.html – the main html file; here you can put font assets, the icon, and page title
- d. client/public/logo.png – image used for logo located on the top left of the page
- e. client/public/mascot.png – image used for the browser tab icon
- f. client/src/components/company.png – image used for navbar logo
- g. client/src/components/Footer.js – code for the page footer
- h. client/src/components/Icon.js – code for styling and routing of icon and company logo
- i. client/src/components/Navbar.js – code for styling and routing of the navbar and page tabs
- j. client/src/components/ScrollTop.js – code for automatic scrollbar behavior
- k. client/src/constants/quiz.js – variables used for the quiz module; in this file, you define quiz constants

- l. `client/src/context/AuthContext.js` – used for authenticating user log in and log out
- m. `client/src/pages/admin/` - all files under this directory is where all the styling and routing of admin operations are held. This includes the topic management, user management, logs, yearly reset, and settings
- n. `client/src/pages/games` – supposedly the directory to put logic files for the game module
- o. `client/src/pages/Achievements.js` – source code for the badges tab
- p. `client/src/pages/Games.js` – source code for the Games tab
- q. `client/src/pages/Home.js` – source code for the homepage or Home tab; includes the styled cards, carousel, and leaderboard
- r. `client/src/pages/Players.js` – source code for the Players tab
- s. `client/src/pages/Profile.js` – source code for the Profile tab; here you can edit the image src file names if you want to change avatar images
- t. `client/src/pages/Register.js` – source code for creating a new account
- u. `client/src/pages/TopicDetail.js` – source code for all quiz routes and logic; the heaviest source code in the frontend. In this file, you can configure the quiz flow for mandatory and bonus questions, and how topics can be displayed in detail.
- v. `client/src/pages/Topics.js` – source code for the training tab
- w. `client/src/pages/UserManual.js` – source code for the user manual
- x. `client/src/styles/retro.css` – main stylesheet reference of all pages
- y. `client/src/utlis/api.js` – the frontend code gateway for communicating with the backend
- z. `client/src/utlis/getImageUrl.js` – code used to allow proper image routing used for badge images and profile avatars
- aa. `client/src/utlis/levelTitles.js` – code for rank logic mostly used for leaderboards and the players tab
- bb. `client/src/App.js` – the main frontend file for compiling and rendering all components
- cc. `client/src/index.js` – file calling the app.js; it's like converting the react app to HTML elements

- dd. client/.env – used to store environment variables used for the frontend and backend communication

Backend Directory (server)

- a. server/middleware/auth.js – code controller for user authentication; mostly used to check if the user is an admin or not when logging in
- b. server/models/ - files under this directory are all the source code for defining user models used in the database. Therefore, adding a new collection means creating a new user model.
- c. server/routes/activity.js – api calls for getting user activities needed for logs
- d. server/routes/admin.js – api calls mainly for admin CRUD operations
- e. server/routes/auth.js – api calls for user authentication, user settings, and user credentials
- f. server/routes/leaderboard.js – api calls for the leaderboard and players tab
- g. server/routes/topics.js – api calls for all logic defined in the frontend codes for the topics and quiz module
- h. server/routes/upload.js - source code for handling custom image uploads used for avatars and badges
- i. server/uploads/badges/ - directory where badge images are stored
- j. server/utils/badgeHelper.js – supplementary code for proper user-badge routing
- k. server/utils/levelSystem.js – utility code for the leveling scheme used for the system; includes the computation logic for the routing whenever a user gains XP
- l. server/utils/levelTitles.js – code logic for defining user ranks or titles found in the players tab
- m. server/utils/logger.js – utility code for logging pre-defined user activities
- n. server/utils/sessionCleanup.js – code for cleaning user sessions every hour
- o. server/.env – file for backend environment variables such as the port, database connection string, web tokens, and frontend url
- p. server/createAdmin.js – a file to manually add an admin user to the database (Note that this file must be deleted in production, but it is a good file backup for development)

- q. server/seedData.js – a file to manually populate topics in the database (Note that this file must also be deleted in production)
- r. server/server.js – the main backend file; stores connection logic and calling all api routes

Collections Directory – A folder containing exported collections of the previously created database. You can import the collections when populating a blank database or when starting from scratch.

VI. Areas for Improvement

Games Module

In the system, there is a games tab that can be developed in order to fully grasp the “gamification” of the system. Part of this module is creating games that is in line with the provided IMS awareness topics. Additionally, add proper XP routing to the game to maintain competence among users.

Quiz Module

The current quiz module has a lot of fixed parameters such as the number of questions, choices, and even question type. This area can be further improved by introducing other types of quizzes such as drag and drop tools, fill-in the blanks, or even text input as answers. Another area for improvement for this module is verifying users if they actually read and watched the reference manual and policy videos provided. This way, a new method of confirming that the user participated in the awareness training can be added.

Topics Module

For the topics module, further development is needed for the dynamic placement of topics reflected upon the client-side pages. In the current state of the system, the topics ISO 9001 and ISO 27001 are bound to be always first in the list of all topics, then followed by topics marked as new, and finally, topics sorted alphabetically. To further improve this, the developer can allow the admin to simply drag the topic cards into the desired order.

User accounts and email verification

Currently, the system allows any email (with proper format) to be registered. An area for improvement is to strictly allow only company emails or valid emails fetched from LDAP. Additionally, when creating new accounts, adding another layer for verification can be added for security purposes.

VII. A Message from the previous Developer

The gamified IMS awareness training was made using Node.js v20.17.0, which was the provided LTS version during the time of development. You can use an updated LTS version for better development. It is important to align proper versions of the stack being used to ensure compatibility and avoid performance issues. The reason is that React.js and Node.js are consistently being updated. Features applied to this project may be deprecated easily over time and it is essential to adjust your code with their updates. Although the project will still run, it is a good practice to reflect your project to updates to avoid deprecation errors, which is common in developing web applications. It may also occur that some dependencies in React may rely on the Node.js version you have installed in your dev environment.

If you know React.js, then well and good. Maybe you can further enhance the project and work on the stated areas for improvement. If, however, you are not familiar with React.js or have little to no experience in JavaScript, then do not worry because you can always start learning. If you are beginning to learn or recollecting your knowledge in JavaScript or any new language, then it is *highly* recommended to **read** books or documentation rather than watching tutorials on youtube or reels. Then, **create small projects** or do exercises to apply what you read. Before you code, it is better to write down logic and make a draft in a piece of paper, because you can visualize your ideas and jot down additional notes you have in mind. Do not rush code. Familiarize yourself with the tools you need before using it. As Abraham Lincoln said “Give me six hours to chop down a tree and I will spend the first hour sharpening the axe”. Fortunately, in our era, we can always find ways to work smart. This brings us to the next message from me to you – proper use of AI.

You are probably reading this in the era of “vibe-coding”. This is not entirely a bad for you as a developer. The greatest advantage for using AI in development is getting

straight-to-the-point answers and even speeding up your learning process. However, please **DO NOT rely on copy-pasting code**. Needless to say that in this era of technology, you are not a true developer if you do not use AI...c o r r e c t l y.

Lastly, if you opt to do a transition/convert the project into another stack, here are some suggestions.

- (1) PERN stack/MySQL/PostgreSQL – you can opt for this stack if you want an easier database structure, but with added complexity. Relational databases are easier to follow and still the most used form of database design, but it is not as flexible as MongoDB.
- (2) Typescript – The latest standard in modern web development and a much more stable version of writing React projects. The current project made use of standard JavaScript framework.
- (3) Java – The main idea for deployment is to integrate the gamified web app to the hosted AWS portal. The portal was made using Java for the backend and JSP for the frontend, which is contradictory to the dynamic nature of a MERN stack that the project was built on. However, if you opt to deploy the application to the portal, then it is best to transition to a similar environment for easier integration. From there, you can just add your project tables to the current database of the portal and even utilize the existing user accounts for easier authentication.
- (4) Tailwind CSS – for easier page styling
- (5) Next.js – a full stack framework, meaning you can create the server-side and client-side without the use of a middleware. However, it is a react-based framework, so you need to learn react.js first if you opt for this one.
- (6) WordPress – only go for this one if the requirements of the project retains the quiz module and no longer require the other gamified schemes. You can create quiz projects that emulates the style of the developed web application using WordPress, but it may be challenging to apply XP routes, leveling, and badge routings for users.